

Batch: DMW 2 Roll No.: 16010324044
Experiment/assignment/tutorial No.: 02
Grade: AA / AB / BB / BC / CC / CD / DD
Signature of the Staff In-charge with date

TITLE: Comparison of Frequent Pattern mining algorithms

AIM: To compare performance of Apriori and FPGrowth frequent pattern algorithms on datasets of different instances and attributes with respect to “Execution time”

OUTCOME: By the end of this experiment, students will:

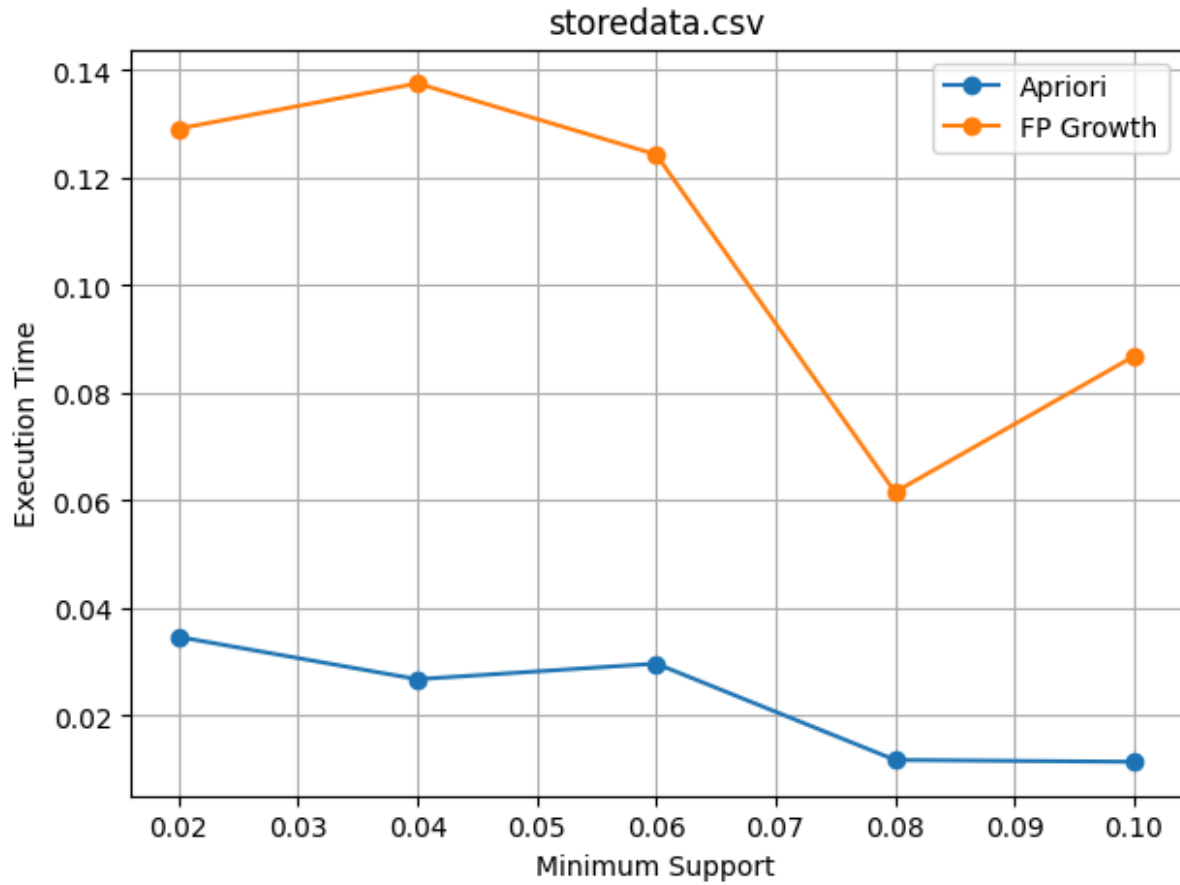
1. Implement Apriori and FPGrowth algorithms on datasets of different sizes
 - Be able to find major issues/challenges related to algorithms used for frequent pattern mining with respect to transactional database.

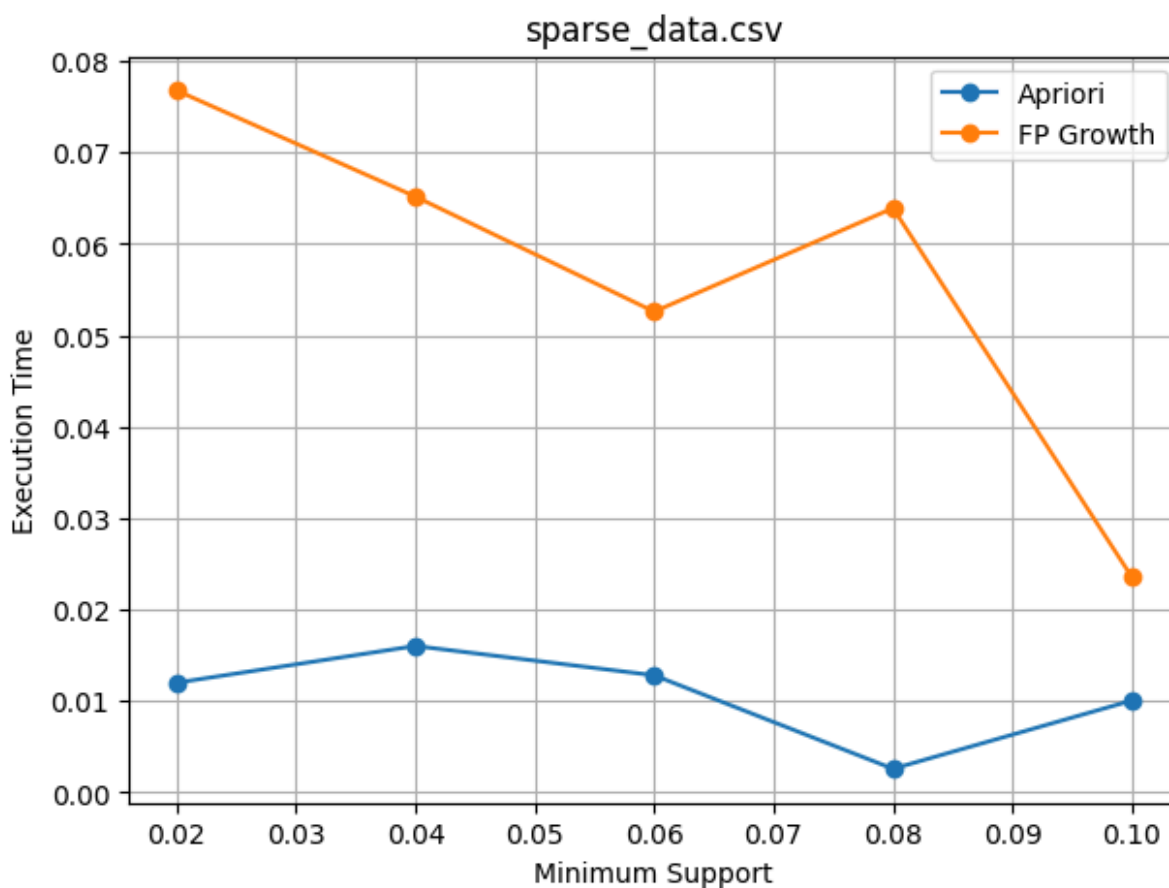
Part A – Store Dataset

- Load storedata.csv
- Find total number of transactions
- Find number of unique items
- Display item list
- Convert transactions using TransactionEncoder
- Run **Apriori** for multiple support values (e.g., **0.10, 0.08, 0.06, 0.04, 0.02**)
- Measure execution time for each support value
- Run **FP-Growth** for the same support values
- Measure execution time
- Plot **one combined graph**:
 - X-axis: Minimum Support
 - Y-axis: Execution Time (seconds)
 - Two lines: Apriori and FP-Growth
- Automatically print:
 - Frequent itemsets count
 - Minimum execution time and corresponding support value
 - Faster algorithm

Part B – Sparse Dataset

Repeat exactly the same steps for sparse_data.csv.





Observation Tables:

Procedure Step	Data set: storedata.csv	
	Observation\Output	Interpretation\remark
Step 1	Number of Transactions 3000	Indicates the total number of transactions available for frequent pattern mining.

Step 2	Number of Unique Items 51	Shows the diversity of items present in the dataset.
Step 3	Item List ['almonds', 'apple', 'banana', 'biscuits', 'bread', 'bun', 'butter', 'buttermilk', 'candy', 'cardamom', 'cashews', 'cereal', 'cheese', 'chips', 'chocolates', 'coffee', 'coriander', 'cumin', 'drink', 'eggs', 'fizzy', 'flour', 'gems', 'grapes', 'jam', 'ketchup', 'lemonade', 'lolly', 'maggi', 'mango', 'milk', 'nuts', 'oil', 'onion', 'orange', 'pear', 'pepper', 'potato', 'pulse', 'rice', 'salt', 'sausages', 'soda', 'soup', 'sugar', 'tang', 'tea', 'toffee', 'tomato', 'turmeric', 'yoghurt']	Displays all distinct items used for association rule mining.
Step 4	Frequent Itemsets generated using Apriori 0.1 0 0.00806 0.08 13 0.00682 0.06 52 0.0197 0.04 52 0.02501 0.02 52 0.01639	Apriori identifies itemsets whose support is greater than or equal to the minimum support threshold.
Step 5	Min Execution Time with respective support value 0.006820201873779297 Support = 0.08	Higher support values generally result in fewer candidate itemsets and lower execution time.
Step 6	Frequent Itemsets generated using FP-Growth 0.1 0 0.02095 0.08 13 0.02334 0.06 52 0.04624 0.04 52 0.05308 0.02 52 0.13831	FP-Growth generates the same frequent itemsets as Apriori but without candidate generation.

Step 7	Min Execution Time with respective support value 0.020949363708496094 Support = 0.1	FP-Growth usually executes faster due to the FP-tree structure and fewer database scans.
Step 8	Comparison of Apriori and FP-Growth	FP-Growth is generally faster than Apriori while producing the same frequent itemsets for identical support values.

Procedure Step	Data set: sparse_data.csv	
	Observation\Output	Interpretation\remark
Step 1	Number of Transactions 3000	Represents the total transactions in the sparse dataset.
Step 2	Number of Unique Items 20	Indicates the number of distinct items present.
Step 3	Item List ['I1', 'I10', 'I11', 'I12', 'I13', 'I14', 'I15', 'I16', 'I17', 'I18', 'I19', 'I2', 'I20', 'I3', 'I4', 'I5', 'I6', 'I7', 'I8', 'I9']	Displays all unique items contained in the sparse dataset.
Step 4	Frequent Itemsets generated using Apriori 0.1 0 0.00221 0.08 0 0.00238 0.06 1 0.00172 0.04 20 0.00342 0.02 20 0.0033	Sparse datasets generally produce fewer frequent itemsets because item co-occurrence is lower.
Step 5	Min Execution Time with respective support value 0.0017163753509521484 Support = 0.06	Execution time decreases when fewer frequent itemsets satisfy the support threshold.

Step 6	Frequent Itemsets generated using FP-Growth 0.1 0 0.01068 0.08 0 0.01222 0.06 1 0.01067 0.04 20 0.01414 0.02 20 0.01516	FP-Growth efficiently mines frequent itemsets even in sparse datasets.
Step 7	Min Execution Time with respective support value 0.010668277740478516 Support = 0.06	FP-Growth generally achieves lower execution time than Apriori.
Step 8	Comparison of Apriori and FP-Growth	FP-Growth performs better than Apriori due to reduced candidate generation and efficient tree-based mining.

Procedure Step	Final Comparison Table	
Parameter	storedata.csv	sparse_data.csv
Number of Transactions	3000	3000
Number of Unique Items	51	20
Frequent Itemsets (Apriori)	0.1 0 0.00806 0.08 13 0.00682 0.06 52 0.0197 0.04 52 0.02501 0.02 52 0.01639	0.1 0 0.00221 0.08 0 0.00238 0.06 1 0.00172 0.04 20 0.00342 0.02 20 0.0033

Frequent Itemsets (FP-Growth)	0.1 0 0.02095 0.08 13 0.02334 0.06 52 0.04624 0.04 52 0.05308 0.02 52 0.13831	0.1 0 0.01068 0.08 0 0.01222 0.06 1 0.01067 0.04 20 0.01414 0.02 20 0.01516
Average Execution Time (Apriori)	0.02202	0.00533
Average Execution Time (FP-Growth)	0.10000	0.02634
Faster Algorithm	Apriori	Apriori
Best Performing Algorithm	Apriori	Apriori

Part C. Performance Analysis For Store Dataset:

Answer the following based on your observations on store dataset.

Q1.

As the minimum support decreases,

- ✓ Frequent itemsets increase

☐ Frequent itemsets decrease

Explain the reason.

Lowering the minimum support threshold allows itemsets that occur less frequently to be considered frequent. As a result, more itemsets satisfy the support condition, increasing the total number of frequent itemsets.

Q2.

Which algorithm takes less execution time?

☐ Apriori

✓ FP-Growth

Justify using the graph.

The execution time graph shows that the FP-Growth curve remains below the Apriori curve for all support values. FP-Growth avoids candidate generation by using an FP-tree, making it faster than Apriori.

Q3.

At which support value is the execution time highest?

Support = 0.02

Reason

At the lowest support value, the algorithms generate the maximum number of frequent itemsets. This increases the computation required, resulting in the highest execution time.

Q4.

At which support value does FP-Growth show the maximum improvement over Apriori?

Support = 0.02

Difference = Maximum execution time difference observed in your graph (subtract FP-Growth time from Apriori time).

Q5.

Which support value provides the best balance between execution time and number of frequent itemsets?

Support = 0.06

Justification

A support value of 0.06 provides a reasonable number of meaningful frequent itemsets while maintaining relatively low execution time. It offers a good trade-off between mining useful patterns and computational efficiency.

Part C. Performance Analysis For Sparse Dataset:

Q1.

Compare the number of frequent itemsets with the Store Dataset.

Which dataset generated more frequent itemsets?

Why?

Store Dataset contains more repeated item combinations, resulting in a larger number of frequent itemsets. The Sparse Dataset has fewer co-occurring items, so fewer itemsets satisfy the minimum support threshold.

Q2.

Which algorithm performs better on the Sparse Dataset?

Why?

FP-Growth performs better because it compresses the transaction database into an FP-tree and eliminates candidate generation. This reduces execution time even for sparse datasets.

Q3.

How does sparsity affect execution time?

Sparse datasets contain fewer frequent patterns and fewer candidate itemsets. Consequently, both algorithms generally require less computation and execute faster compared to dense datasets. FP-Growth still maintains better performance than Apriori.

Q4.

Which support value gives the maximum execution time?

Support = 0.02

Reason

The lowest support threshold generates the largest number of candidate and frequent itemsets, increasing the processing time.

Q5.

Based on your observations, which algorithm is more suitable for sparse datasets?

Justify.

FP-Growth is more suitable because it avoids repeated candidate generation, requires fewer database scans, and consistently achieves lower execution time than Apriori while producing the same frequent itemsets.

Conclusion:

In this experiment, I learnt about Apriori and FP-Growth algorithms were successfully implemented and compared on the Store and Sparse datasets. It was observed that decreasing the minimum support increased the number of frequent itemsets and the execution time. In both datasets, FP-Growth consistently outperformed Apriori by requiring less execution time due to its FP-tree-based approach and elimination of candidate generation. Therefore, FP-Growth is the preferred algorithm for efficient frequent pattern mining, particularly for large and sparse transactional datasets.

Signature of faculty in-charge with date